

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	M Vishnu Vardhan	Reg. No.:	192472087
Section / Batch:	2024	Date:	27-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab
DAA PROGRAMMING


**MANCHIKALAPATI VISHNU
VARDHAN**
TS24730879C

CO5 AT1-Constraint-Based Problem Solving
Back

Language: Python C C++ Java

QUESTIONS

Q1
Steiner Tree Problem (NP-Hard / Constraint Optimization Problem)
100 marks

1/1 answered

Q1 - Steiner Tree Problem (NP-Hard / Constraint Optimization Problem)
100 marks

Given a graph with required terminal vertices, determine the minimum-cost tree connecting them. Clearly define constraints such as connectivity and inclusion of required nodes. Analyze how constraints influence the search space and solution complexity. Develop a backtracking-based solution to explore feasible trees. Apply pruning techniques to eliminate non-optimal trees. Justify how your solution ensures minimal cost and discuss NP-Hard complexity.

Your Code

```

1 import java.util.*;
2
3 public class SteinerTree {
4     static int V, E;
5     static int[][] graph;
6     static int INF = Integer.MAX_VALUE / 2;
7     static int minCost = INF;
8     static int[] parent;
9     static boolean[] selectedEdge;
10
11     public static void main(String[] args) {
12         Scanner sc = new Scanner(System.in);
13         V = sc.nextInt();
14         E = sc.nextInt();
15         graph = new int[V][V];
16         for (int[] row : graph) Arrays.fill(row, INF);
17         for (int i = 0; i < V; i++) graph[i][i] = 0;

```

Input

```

5 6
0 1 2
0 2 3
1 2 1
...

```

Output

```

8

```

Steiner Tree Problem — simple steps

1. **Read the map** — list all the points (vertices) and paths (edges) between them, each path has a cost.
2. **Mark the "must-connect" points** — some points are required; they all need to end up joined together.
3. **Start picking paths one at a time** — try adding an edge that helps connect the required points.
4. **Check two things before keeping a path:**
 - Does it avoid creating a loop?
 - Is the total cost so far still better than the best answer found already?
5. **If a choice breaks a rule or costs too much** — undo it and try a different path instead.
6. **If a full set of paths connects all required points** — compare its total cost to the cheapest one found so far, and keep whichever is lower.
7. **Repeat until every reasonable combination has been tried.**
8. **Print the lowest cost found** — that's the final answer (8 in this case).